CHAPTER 4

DESIGN

4.1 Architecture: MVVM (Model-View-ViewModel)

In our project, we have adopted the MVVM architecture pattern, which ensures clear separation of concerns within the application. The MVVM pattern consists of three primary components:

- **Model:** Represents the data and business logic.
- **View:** Represents the UI, which displays the data.
- **ViewModel:** Acts as the intermediary between the View and the Model, handling the business logic and preparing the data for presentation.

By adopting **MVVM**, we achieve the following:

- **Separation of Concerns:** The UI and business logic are separated, making the app easier to maintain and test.
- **Unit Testing and Code Reusability:** ViewModels facilitate easier unit testing and enable reusable logic across the app, ensuring high-quality code.
- **Scalability:** As the project scales with features like ML models or customized user preferences, MVVM supports seamless adaptation and expansion.

4.2 System Design

Our system is designed to be modular, efficient, and scalable, with distinct layers and components for each part of the application.

Layers and Modules

1. **Presentation Layer:**
    - This layer includes Activities, Fragments, and Flutter Widgets, which are responsible for displaying data to the user and receiving user input. The XML layouts and Flutter UI components define the interface.
    - **Role in Our Project:** The presentation layer will handle the display of ML-based recommendations, mood detection results, and virtual try-on functionality.

2. **Domain Layer:**
    - The **Domain Layer** includes the **business logic** and the **use cases**. This layer processes data received from the ViewModel and performs necessary actions, such

as fetching outfit recommendations, handling user preferences, or managing mood-based suggestions.

- **Role in Our Project:** This layer is where the core business logic for personalized recommendations, mood analysis, and virtual try-on resides.

3. **Data Layer:**
   - The **Data Layer** consists of **repositories** and **data sources**, which manage local and remote data operations. It handles fetching data from the database and interacting with external APIs (e.g., for fetching outfit data).
   - **Role in Our Project:** The data layer will manage the user's preferences and past interactions, which will be crucial for personalizing the system's suggestions. It will also fetch outfit data and interact with backend APIs to support the real-time functionality of the virtual try-on feature.

4.3 Technology Stack

1. **Language:** Python
   - Python is used for its simplicity, readability, and extensive libraries, making it ideal for rapid development and integration with Flask for the backend.
2. **Backend Framework:** Flask
   - Flask is a lightweight and flexible framework for building web applications and APIs. It supports rapid development and integrates well with Python.
3. **IDE:** Visual Studio Code
   - Visual Studio Code is a versatile code editor with extensions and debugging tools that support Python development.
4. **Database**: SQLite
   - SQLite is used for its simplicity and lightweight structure, making it suitable for storing user preferences, mood history, outfit data, and other essential information.
5. **Cloud Services:** AWS
   - AWS provides scalable cloud infrastructure for hosting the backend, database, and APIs. Services like AWS EC2 and S3 support real-time data storage and retrieval.
6. **Network:** RESTful APIs
   - RESTful APIs will facilitate communication between the frontend and backend, ensuring efficient data exchange for recommendations and virtual try-on functionality.

7. **Dependency Injection:** Hilt
    - Hilt will simplify dependency injection by providing automatic, easy-to-use mechanisms for injecting dependencies (e.g., APIs, databases) throughout our application.
8. **Asynchronous Programming:** Coroutines
    - Coroutines will be used for managing background tasks, such as fetching user data, mood analysis, and virtual try-on processing. This ensures that the app remains responsive by performing heavy tasks off the main UI thread.
9. **Testing:** Pytest, JUnit, Espresso
    - Pytest will be used for unit testing Python-based backend functions.
    - JUnit will be used for unit testing our View Models, business logic, and data-related functions.
    - Espresso will help us in UI testing, ensuring that the Activities and Fragments display data correctly and respond to user inputs appropriately.

4.4 Key Components

1. **Models:**
    - The Models represent the data structure in the app (e.g., Task, User). These models are used throughout the application to maintain a consistent structure for data.
    - Role in Our Project: Models will represent the user's preferences, mood data, and outfit details. They will interact with the data layer and ViewModel to provide relevant information to the UI.

2. **Views:**
    - Views are composed of Activities, Fragments, and XML layouts, which together define the app's UI.
    - Role in Our Project: The views will display ML-based outfit recommendations, handle interactions with the user for mood detection, and showcase the virtual try-on feature.

3. **ViewModels:**
    - ViewModels are responsible for preparing and managing the data for the Views. They handle user input, interact with the domain layer, and prepare the necessary data for display.
    - Role in Our Project: ViewModels will manage the logic for fetching outfit

recommendations, interacting with the mood detection system, and coordinating the virtual try-on feature.

4.5 API and Backend Integration

- **API:** RESTful services using Flask will facilitate communication with the backend.
- **Endpoints:**
  - Tasks for CRUD operations on tasks and managing the system's data.
  - Users for user management, including storing preferences and historical interaction data.
- **Role in Our Project:** The API will interact with the cloud-based backend, providing real-time updates and syncing user data to improve personalized suggestions.

4.6 Dependency Management

- **Libraries Used:**
  - Flask for backend development.
  - SQLAlchemy for database interactions.
  - Requests for making HTTP requests
  - Hilt for dependency injection.
  - Gson for JSON parsing between the app and backend.

4.7 Security Considerations

- Secure API Communication: We will use HTTPS to encrypt API communications and ensure user data is securely transmitted.
- AWS Security: AWS Identity and Access Management (IAM) will manage secure access to cloud resources.
- Encrypted Shared Preferences: For securely storing sensitive data, such as user preferences or authentication tokens

4.8 Testing Strategy

- Unit Testing: We'll use Pytest for backend logic and JUnit to test ViewModels and use cases to ensure that the business logic works correctly.
- UI Testing: Espresso will be used to test user interfaces, ensuring that the application displays content correctly and handles user interaction properly.

- Integration Testing: MockWebServer will be used to simulate API responses and test how the app handles different scenarios.

4.9 Flowchart

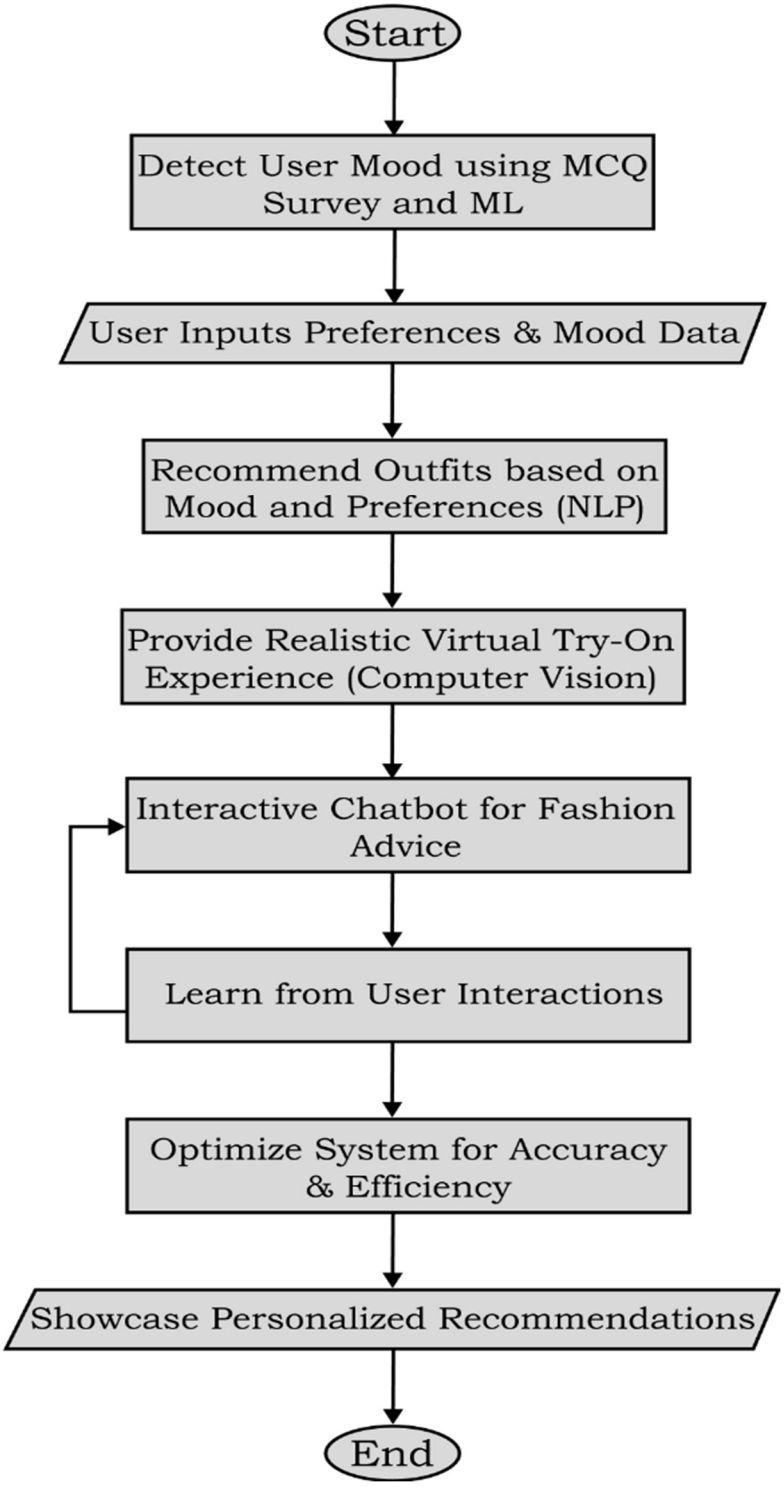

Figure 4.1: Virtual Stylist and Outfit Try on Flowchart